

Programação – Exame da Época Normal

15 de junho de 2026 – Duração: 120 minutos

LEI, LEI-PL, LEI-CE

1. Considere as seguintes definições:

```
struct autor {  
    char nome[100];           // Nome do autor  
    int id;                   // Identificador único do autor  
    int nLivros;              // Número de livros comercializados deste autor  
};  
  
struct livro {  
    char titulo[100];         // Título do livro  
    int idAutor;              // Identificador do autor do livro  
};
```

Uma livraria armazena num ficheiro binário informação sobre os autores e respetivos livros que tem em stock. O ficheiro tem a seguinte organização:

- No início encontra-se um valor inteiro que indica quantos autores são comercializados.
- A seguir a esta informação, para cada um dos autores, surge a seguinte informação: uma estrutura do tipo *struct autor* contendo informação básica sobre o autor e várias estruturas do tipo *struct livro*, uma para cada um dos livros deste autor. O número de livros de cada autor pode ser obtido no campo *nLivros* da estrutura *struct autor*.

Ao lado pode consultar um exemplo de um ficheiro com 2 autores. O primeiro autor, Artur Silva com identificador 5, tem 2 livros no catálogo. As estruturas dos 2 livros deste autor surgem imediatamente após a informação armazenada na *struct autor*. O segundo autor é a Ana Ponte e tem 3 livros.

Dentro do ficheiro, os autores estão ordenados pelo valor do *id*. Os livros de cada autor não estão ordenados por nenhum critério.

livros.bin



Escreva uma função em C que adicione um livro ao ficheiro. Pode assumir que já foi efetuada uma validação prévia e que o autor deste novo livro já se encontra no ficheiro. A função deve apenas adicionar a estrutura do tipo *struct livro* relativa ao novo livro (no local adequado). A função recebe, como parâmetros, o nome do ficheiro a atualizar, o título do novo livro e o identificador do autor. Devolve 1 se a atualização for efetuada com sucesso, ou 0, caso exista algum problema no processamento do ficheiro.

[Cotação: 30%]

2. Considere as seguintes definições:

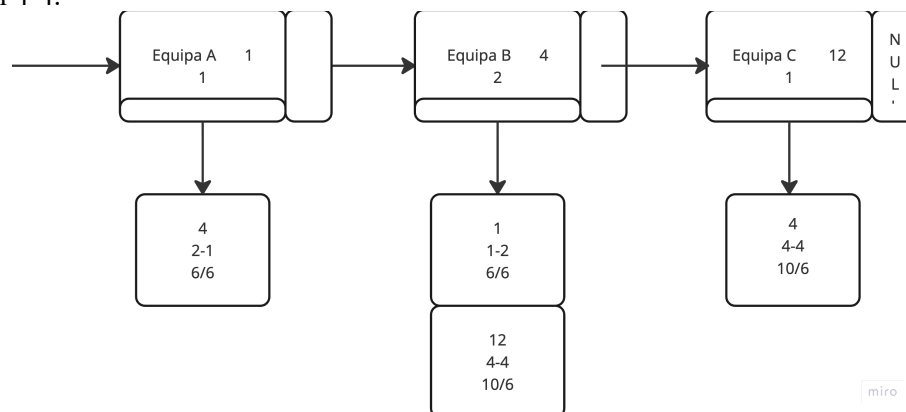
```
struct data {int dia, mes;};

typedef struct {
    int adversario;           // Identificador do adversário
    int m, s;                 // Golos marcados e sofridos
    struct data d;            // Data do jogo
} jogo;

typedef struct equipa no, *pno;
struct equipa {
    char nome[100];           // Nome da equipa
    int id;                   // Identificador numérico único da equipa
    jogo *tab;                // Ponteiro para a tabela de jogos
    int tam;                  // Número de jogos realizados
    pno prox;
};
```

Os dados sobre os jogos realizados num torneio de futebol estão armazenados numa estrutura dinâmica. Existe uma lista ligada simples constituída por nós do tipo *no*, um para cada uma das equipas que participam no torneio. Cada equipa é identificada pelo seu *id* numérico único e a lista está ordenada de forma crescente por este critério. Um vetor dinâmico de estruturas do tipo *jogo* está associado a cada equipa. Este vetor serve para armazenar informação sobre os jogos já realizados por essa equipa. Para cada jogo é identificado o *id* do adversário, o resultado (golos marcados e sofridos) e a data de realização. O ponteiro *tab* na estrutura da equipa permite aceder ao seu vetor de jogos e a dimensão da tabela está armazenada no campo *tam*.

A figura exemplifica uma situação em que existem 3 equipas, com ids 1, 4 e 12. A equipa com id 4 realizou 2 jogos: no primeiro contra o adversário com id 1, o resultado foi 1-2. No segundo, contra o adversário 12, o resultado foi 4-4.



Escreva uma função em C descubra quantos jogos foram realizados numa determinada data. A função recebe, como parâmetros, o endereço inicial da lista e a data a pesquisar. Devolve o número de jogos que se realizaram nessa data.

[Cotação: 25%]

3. Escreva uma função em C que adicione um jogo à estrutura dinâmica. O protótipo da função é o seguinte:

```
int novoJogo(pno lista, int idX, int idY, int mX, int sX, struct data d);
```

A função recebe, como parâmetros, o endereço inicial da lista, os identificadores das equipas *idX* e *idY* envolvidas no jogo, o número de golos marcados e sofridos pela equipa com identificador *idX* e a data em que o jogo se realizou. Esta informação deve ser adicionada aos vetores dinâmicos das 2 equipas envolvidas. Devolve 1 se a atualização for efetuada com sucesso, 0 se alguma das equipas não existir na lista ligada, ou -1, em caso de erro de gestão dinâmica.

[Cotação: 35%]

Programação – Exame da Época Normal

15 de junho de 2026 – Duração: 120 minutos

LEI, LEI-PL, LEI-CE

Nome: _____ Número: _____

4. Funções Recursivas

Qual é o output na consola quando é executado o seguinte programa em C?

```
#include <stdio.h>

int f1(int a[], int tam) {
    if (tam <= 2)
        return 0;
    else {
        printf("%d\t", a[0]);
        return a[0] + f1(a+2, tam-2);
    }
}

int main() {
    int tab[6] = {10,20,30,40,50,60};

    printf("%d\n", f1(tab, 6));
    return 0;
}
```

Resposta:

☐	10	10	
☐	10	30	
☐	10	20	
☐	10	30	40
☐	10	20	40
☐	10	10	40

[Cotação: 10%] (uma resposta errada não desconta)